

Presented to JPD Garage

For the year **2022-2023**

This document is a deliverable agreed upon by the Product Owner

And the development team for the project

in fulfillment of CSSWENG

JPD Garage: Online Management System

Submitted by:

Bernardo, Noah

De Leon, Francis Zaccharie

Dela Cruz, Frances Julianne

Gabini, Brian

Gamboa, Mikkel Dominic

Muriel, Gabriel

Verano, Carl Matthew

Villamiel, Denis Leeroi

Yu, Hanz Patrick

Submitted to:

[redacted]

January 7, 2024

Table of Contents

1. Introduction **3**
1.1. Project Overview 3
1.2 Identified Problems 3
1.3. Software Development Methodology 3
1.4. Stakeholders 4
2. System Architecture and Design **7**
2.1. Key Features 7
2.2. User Flow 9
2.2.1 Partsman Employee Scenario 9
2.2.2 Secretary Employee Scenario 10
2.2.3 Administrator Employee Scenario 11
2.3. Data Design 11
2.3.1. Cart Schema 11
2.3.2. Inventory Item Schema 12
2.3.3. Order Schema 12
2.3.4. User Model 13
2.4. User Interface 13
2.4.1. General Design Choices 14
2.4.2. Dashboard 15
2.4.3. Admin Accessible Pages 16
2.4.4. Inventory 18
2.4.5. Sales and Payments 20
2.4.6. Check Out 23
2.4.7. Audit Log 25
3. Feature Development **26**
3.1. Tech Stack 26
3.2. Sprint Iteration 1 28
3.3. Sprint Iteration 2 30
3.4. Sprint Iteration 3 32
4. Quality Assurance and Testing **34**
4.1. Test tools 34
4.2. Test Approach 35
4.3. Sprint Iteration 1 36
4.4 Sprint Iteration 2 38
4.5 Sprint Iteration 3 40
5. Team Collaboration and Communication **42**
5.1. Collaborative Tools 42
5.2. Practices 42
5.3. Product Owner and Client Communication 43
6. Future Roadmap and Recommendations **44**
6.1 Unimplemented Features 44
6.2 Recommendations 44

1. Introduction

The project is proposed to JPD Garage in fulfillment of the course “Introduction to Software Engineering”

1.1. Project Overview

The JPD Garage: Online Management System aims to streamline and enhance the management of motorcycle parts inventory. The focus lies on creating a straightforward checkout system, enabling efficient addition and removal of stocks for various motorcycle components. The system will facilitate the sale of these parts, and to ease the restocking process to maintain proper inventory stocks. Furthermore, the software will compile a comprehensive list of sales transactions, providing a transparent overview of all transactions made.

1.2 Identified Problems

The challenge faced by the product owner at JPD Garage is the reliance on manual Excel-based processes for managing the list of sales and stocks. This cumbersome method requires the secretary to painstakingly add and remove sales entries, as well as verify stock availability. This manual approach not only consumes valuable time but also introduces the risk of errors and discrepancies in the records. The need for a more efficient and automated solution is evident, as the existing system hampers productivity and accuracy. The project aims to address this issue by providing a streamlined and user-friendly platform, ultimately alleviating the burden of manual data entry and ensuring a more seamless management of sales and stock transactions.

1.3. Software Development Methodology

The team conducted 3 Sprint Iterations with the Scrum Agile Methodology in order to accomplish the project. Agile is a set of principles that focuses on quickly and efficiently producing workable products to the product owner while minimizing the least amount of documentation. Furthermore, Scrum is a

methodology under agile that allows teams to “self-organize” and take other practices from other frameworks when needed. This framework was chosen not only for its versatility, but it is well-suited in practices where business problems are identified, while solutions can be iteratively worked on to solve challenges encountered.

1.4. Stakeholders

Key Business Stakeholders of JPD Garage

Name	*Role*
[redacted]	*CEO / Owner*

Project Responsibilities

Role	Identity	Responsibilities	Contact Details
Scrum Master	Yu, Hanz Patrick	Scrum Activity Facilitator, Remove Impediments, Scrum Board Administration	hanz_yu@dlsu.edu.ph
Business Analyst	Muriel, Gabriel	Requirements Elicitation and Analysis, Communication, Process Improvement and Optimization	gabriel_muriel@dlsu.edu.ph
Frontend Developer	Dela Cruz, Frances Julianne	User Interface (UI) Design and Development,	francesjuliannedelacruz@gmail.com

		Collaboration with Design and Backend Teams	
Frontend Developer	Villamiel, Denis Leeroi	User Interface (UI) Design and Development, Collaboration with Design and Backend Teams	denis_leeroi_villamiel@dlsu.edu.ph
Backend Developer	Bernardo, Noah	Server-Side Logic and Database Management, API Development and Integration, Performance Optimization	noah_h_bernardo@dlsu.edu.ph
Backend Developer	Gabini, Brian	Server-Side Logic and Database Management, API Development and Integration, Performance Optimization	briangabini@gmail.com
Backend Developer	Verano, Carl Matthew	Server-Side Logic and Database Management, API Development and Integration, Performance Optimization	carl_verano@dlsu.edu.ph

Quality Assurance	De Leon, Francis Zaccharie	Test Planning and Strategy, Test Case Design and Execution, Defect Tracking and Reporting	francis_zaccharie_deleon@dlsu.edu.ph
Quality Assurance	Gamboa, Mikkel Dominic	Test Planning and Strategy, Test Case Design and Execution, Defect Tracking and Reporting	mvdgamboa@gmail.com

2. System Architecture and Design

This section contains the project's infrastructure encompassing both the User Interface and Database Design Schemas.

2.1. *Key Features*

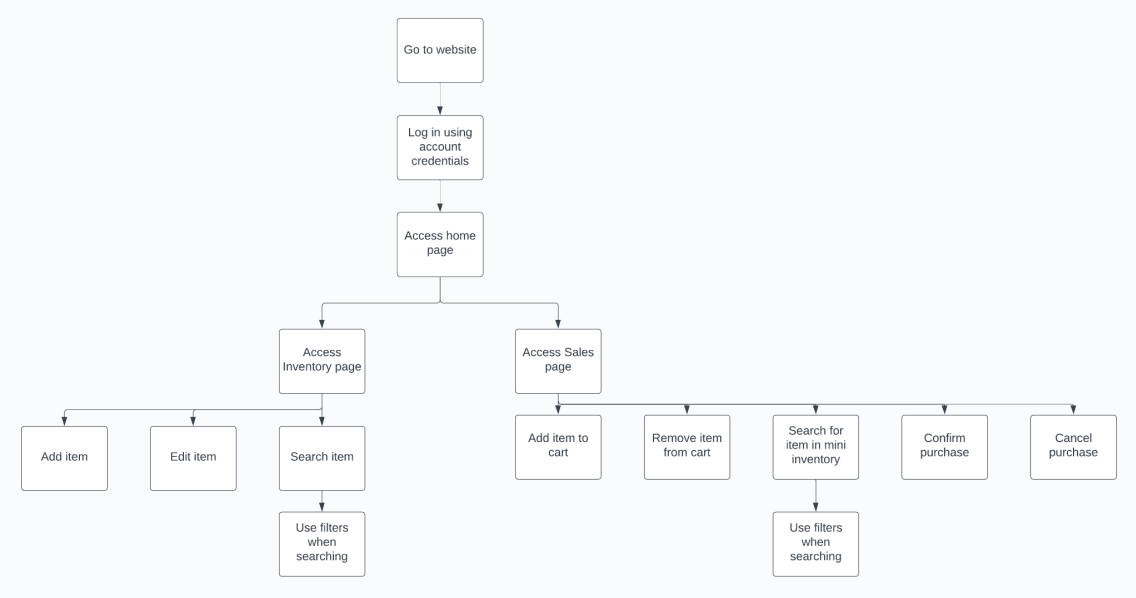
Feature	*Description*
Account Creation and Management	Allows administrators to create accounts, granting employees access to the website.
Account Visibility	Allows any employee to have access to the website with varying degrees of permissions and limitations appropriate to their role within the company.
Log-In Page	Allows any employee to input their account credentials in order to securely access the website.
Inventory - Add	Allows partsman employees to update the inventory by adding newly-acquired products.
Inventory - Edit	Allows partsman employees to update the inventory by editing product information, such as, but not limited to: item name, stock quantity, and retail price.
Inventory - Delete	Allows administrator employees to

	update the inventory by deleting products that no longer need to be tracked from the inventory.
Inventory - Search	Allows partsman employees to search for products that exist in the inventory.
Inventory - Filter	Allows partsman employees to filter their searches based on certain criteria.
Inventory - Downloading and Printing	Allows administrator employees to download and print the .csv version of the inventory on the website for accounting and regulatory purposes.
Sales - Summary	Allows administrator and secretary employees to view a substantial summary of financial information related to the company's product sales.
Sales - Mini Inventory	Allows any employee to view a minimized version of the inventory beside the list of items in the cart at the check-out page.
Sales - Add to Cart	Allows any employee to choose and add an item to the cart, which can be checked out later given the proper transaction.
Sales - Remove from Cart	Allows any employee to choose and

	remove an item from the cart should said item/s no longer need to be checked out.
Sales - Searching and Filtering	Allows any employee to search for—and narrow down through filtering—any item in the mini inventory that they may wish to add to the cart.
Sales - Purchase	Allows any employee to confirm a purchase or otherwise cancel a purchase and reset the items in the cart.

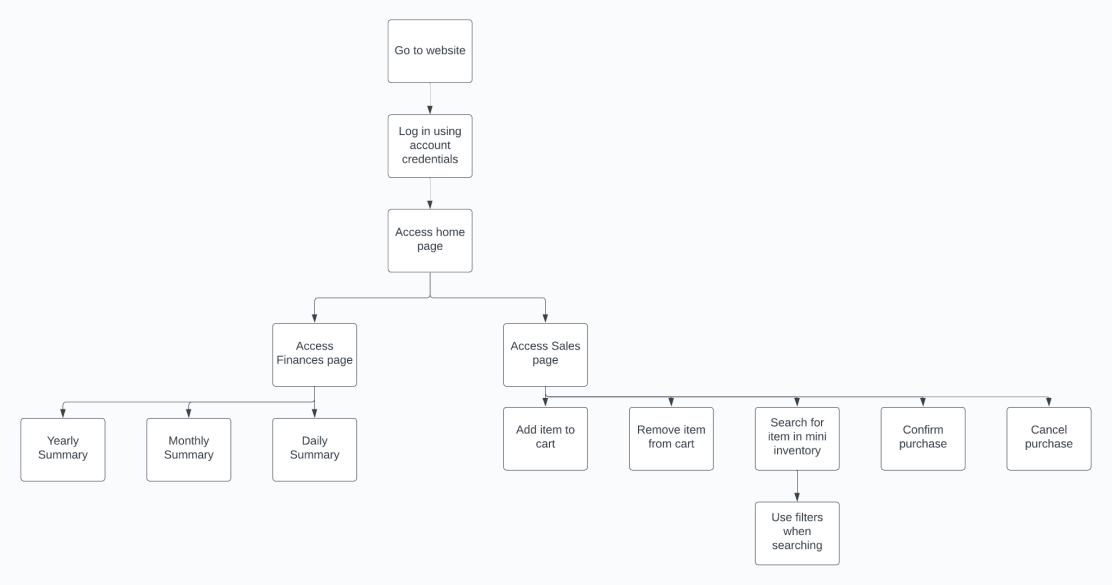
2.2. User Flow

2.2.1 Partsman Employee Scenario



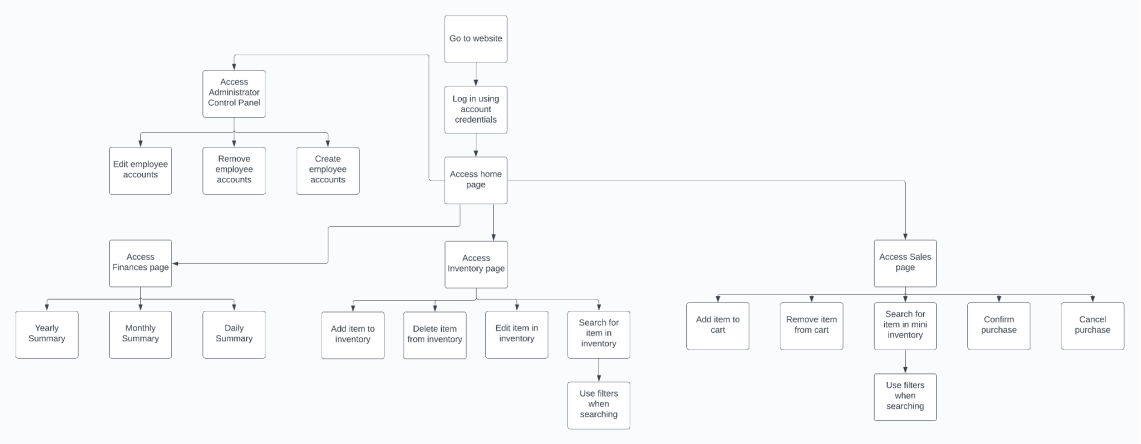
The Partsman Employee Activity Diagram above shows the flow of the processes and actions that may be taken when a partsman accesses the website. Partsman employees do not have access to the Finances page and are not allowed to delete items from the inventory.

2.2.2 Secretary Employee Scenario



The Secretary Employee Activity Diagram above shows the flow of the processes and actions that may be taken when a secretary accesses the website. Secretary employees do not have access to the Inventory page but are allowed to access the Finances page. They still retain access to the Sales page.

2.2.3 Administrator Employee Scenario



The Administrator Employee Activity Diagram above shows the flow of the processes and actions that may be taken when an administrator accesses the website. Administrators are allowed to access all pages of the website, along with an extra hidden page for administrators that allows them to add, edit, and remove employee accounts from the website.

2.3. Data Design

2.3.1. Cart Schema

Field	*Type*	*Description*
user	ObjectId (Reference: User)	Unique identifier for the user associated with the cart.
inventoryItems	Array of Objects	List of inventory items and their quantities in the cart.
transactionType	String (Enum)	Type of transaction (retail or wholesale).
totalPrice	Number	Total price of items in the cart.

This schema serves as the blueprint for the cart functionality within the application. It facilitates the association of users with their respective carts,

manages the inventory items added to the cart along with their quantities, specifies the transaction type, and calculates the total price of items in the cart.

2.3.2. Inventory Item Schema

Field	*Type*	*Description*
partName	String	Name of the inventory item.
brand	String	Brand of the inventory item.
motorModel	String	Motor model associated with the inventory item.
stockNumber	Number	Quantity of the item in stock.
wholesalePrice	Number	Wholesale price of the item.
retailPrice	Number	Retail price of the item.
stockStatus	String (Enum)	Current stock status (In Stock, Out of Stock, Danger Zone).
dateAdded	Date	Timestamp for when the item was added.
dateModified	Date	Timestamp for when the item was last modified.

This schema outlines the structure for managing inventory items within the application. It includes essential details such as the part name, brand, motor model, stock quantity, wholesale and retail prices, stock status, and timestamps for tracking when the item was added or modified.

2.3.3. Order Schema

Field	*Type*	*Description*

items	Array of Objects	List of items in the order with their details.
totalPrice	Number	Total price of items in the order.
transactionType	String (Enum)	Type of transaction (retail or wholesale).
orderDate	Date	Date and time when the order was placed.

This schema defines the structure for managing orders within the application. It encompasses details about the items included in the order, their respective prices and quantities, the total order price, the transaction type (retail or wholesale), and the date when the order was placed.

2.3.4. User Model

Field	*Type*	*Description*
items	Array of Objects	List of items in the order with their details.
totalPrice	Number	Total price of items in the order.
transactionType	String (Enum)	Type of transaction (retail or wholesale).
orderDate	Date	Date and time when the order was placed.

This schema acts as the foundation for managing user information within the application. It includes details such as the user's email (unique identifier), password, employee name, role (Partsman, Cashier, Secretary, Admin), and timestamps for tracking when the user was created or last modified.

2.4. User Interface

2.4.1. General Design Choices

- Overall Theme and Aesthetics
    - For the aesthetics of the website, the client requested for red to be the main color used and that the design should follow a simple layout. Keeping this in mind, the designers decided to utilize “Cards” in order to efficiently organize information in a way that would allow the user to focus on the information presented.
- Separate Dashboard for Employees and Admins
    - Multiple people are able to use the website in which they all have different responsibilities. Having separate dashboards would allow them to easily access the information they need. It also restricts the information that they cannot have access to.
- Header Bar
    - The header bar contains the title of the website, the user who is currently accessing the website and the button to access the shopping cart. They are placed here for easier access no matter the page the user is on.
- Side Bar
    - The side bar consists of buttons to easily navigate to the Inventory, Finance, Add Item, Check out, and Audit Log. Additionally, to conserve space for more important information, it can be expanded and minimized.
- Notification Bar
    - The Notification Bar at the side allows users to quickly check the updates done. Moreover, the icons allow the users to quickly identify what kind of update it is.
- Deletion/Confirmation/Viewing of Individual Information
    - Modals are used for confirmations, deletion and viewing of individual information. This allows the users to quickly switch from the general list of items to the individual item and vice versa.

2.4.2. Dashboard

The layouts for the dashboard which includes how it would look like with the sidebar minimized and expanded. Additionally, it includes how an individual notification would show up once clicked.

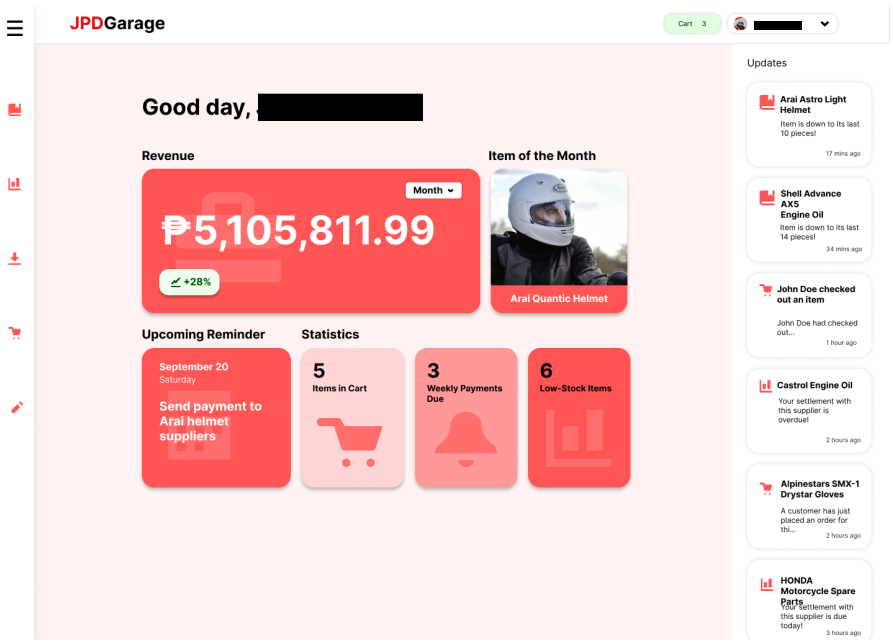


Figure 1. Dashboard with minimized sidebar

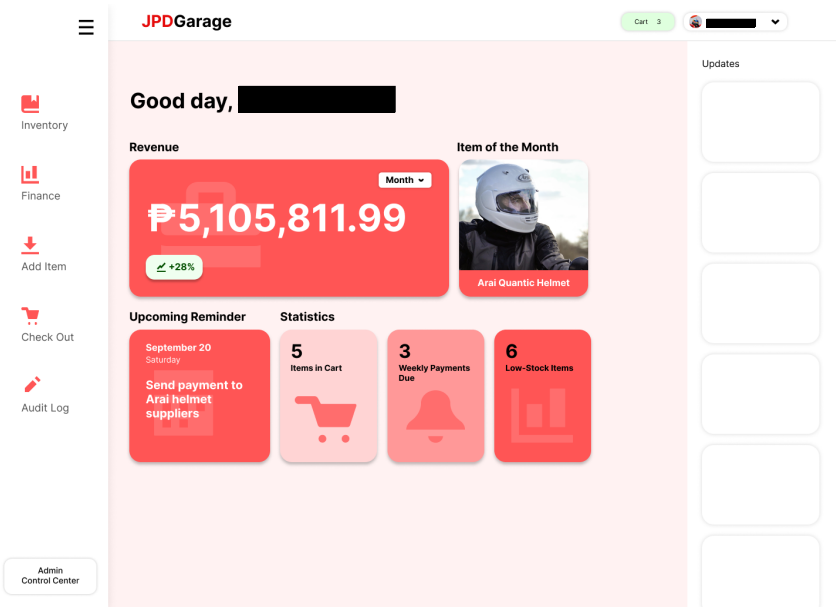


Figure 2. Dashboard with expanded sidebar

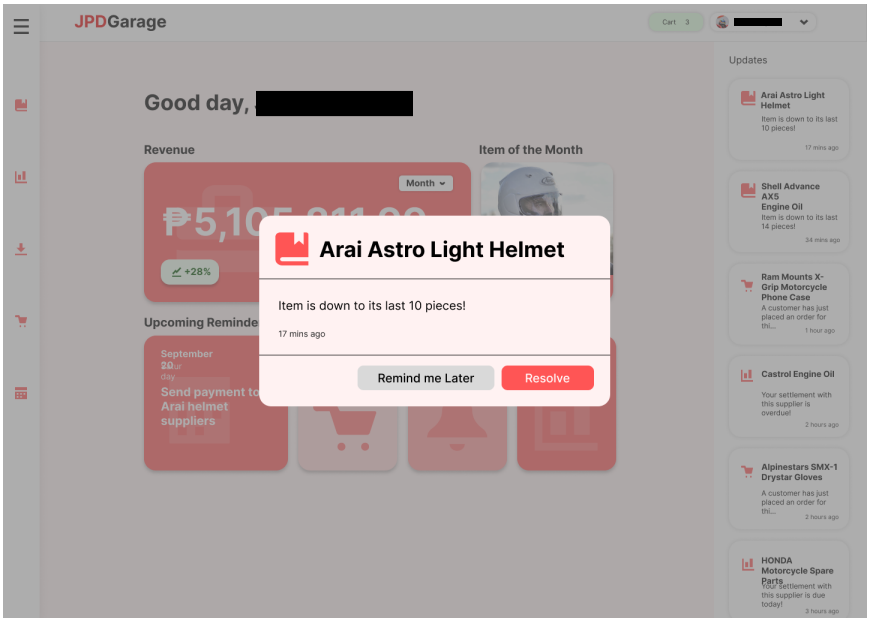


Figure 3. Dashboard with Notification Modal

2.4.3. Admin Accessible Pages

The layouts for the dashboard made for a user who is an administrator. It includes different pages than the first dashboard. It includes the verified list of users and the actions of the users in the website.

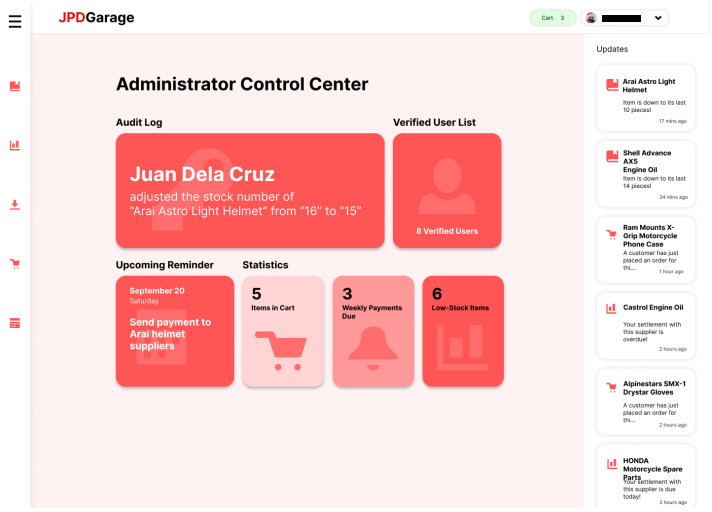


Figure 4. Admin Dashboard

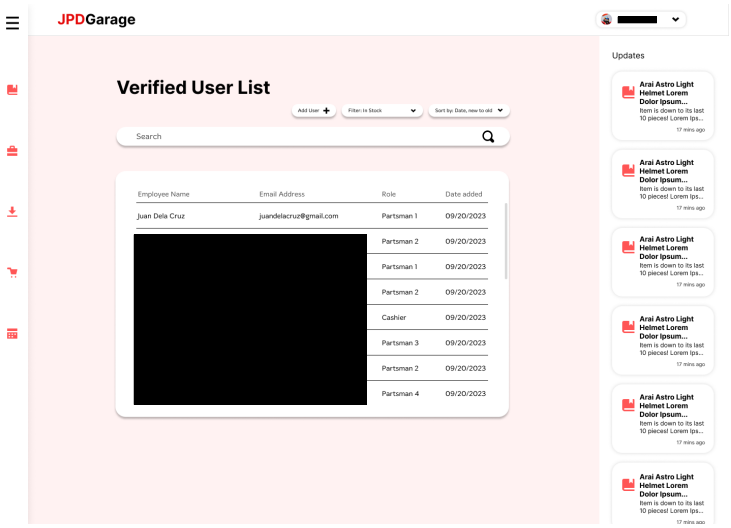


Figure 5. Verified Users List

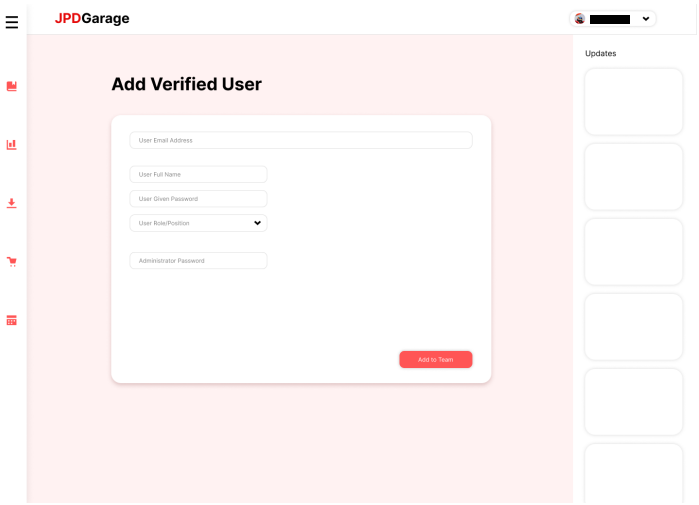


Figure 6. Add Verified User

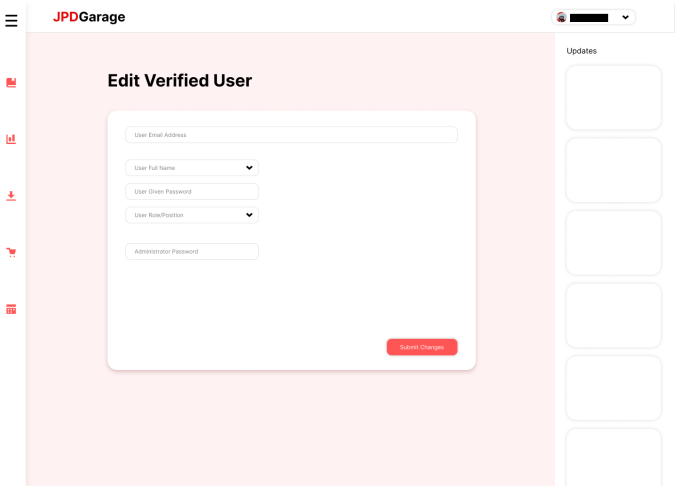


Figure 7. Edit Verified User

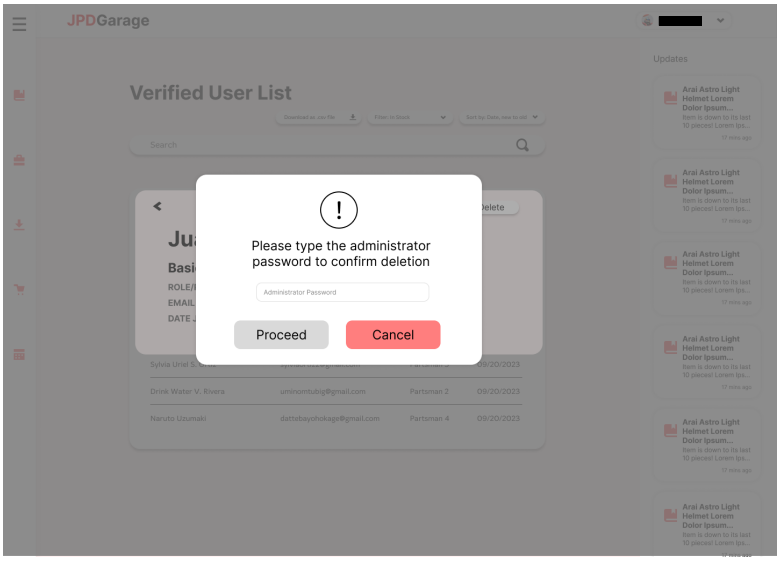


Figure 8. Delete Verified User

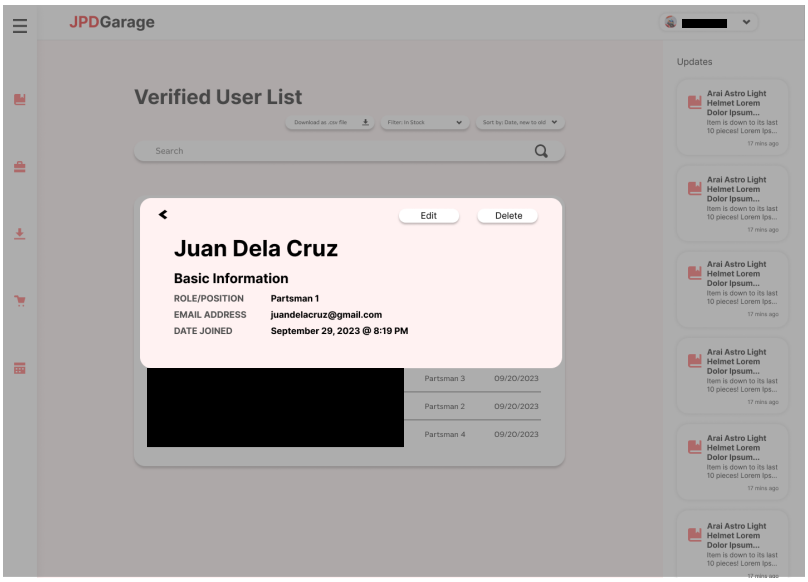


Figure 9. Verified User

2.4.4. Inventory

The layouts to see the items currently in the inventory.

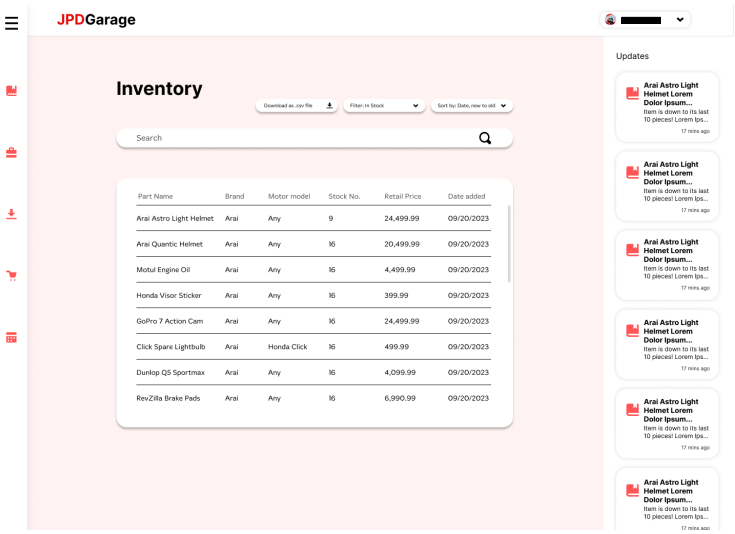


Figure 10. Inventory

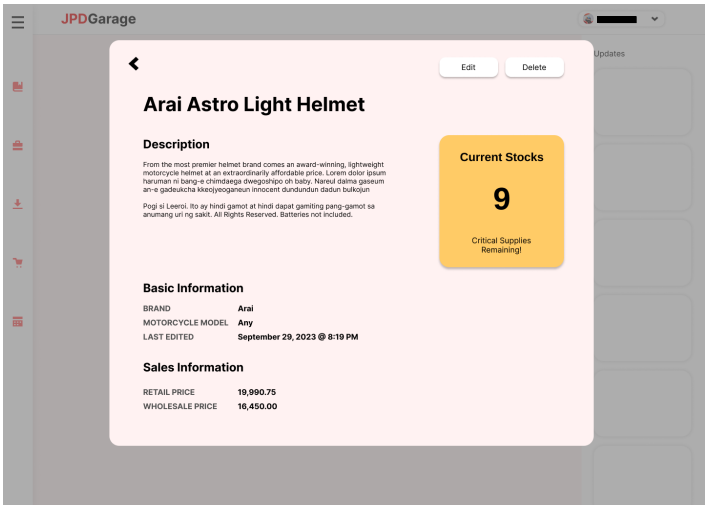


Figure 11. Inventory Item

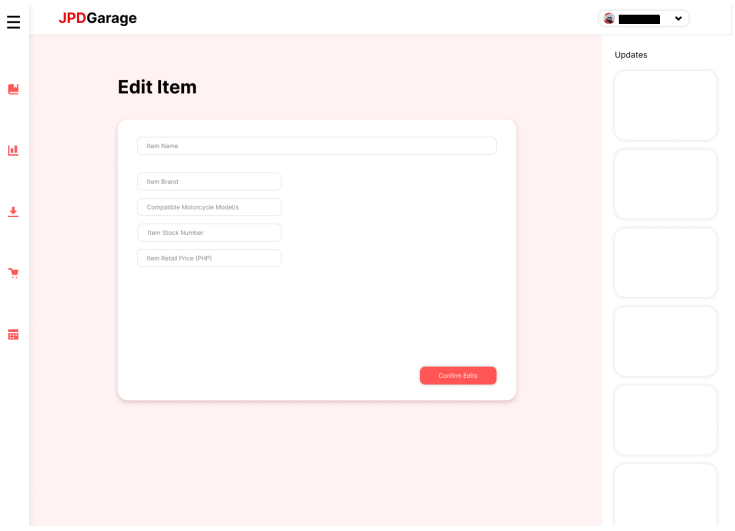


Figure 12. Edit Item

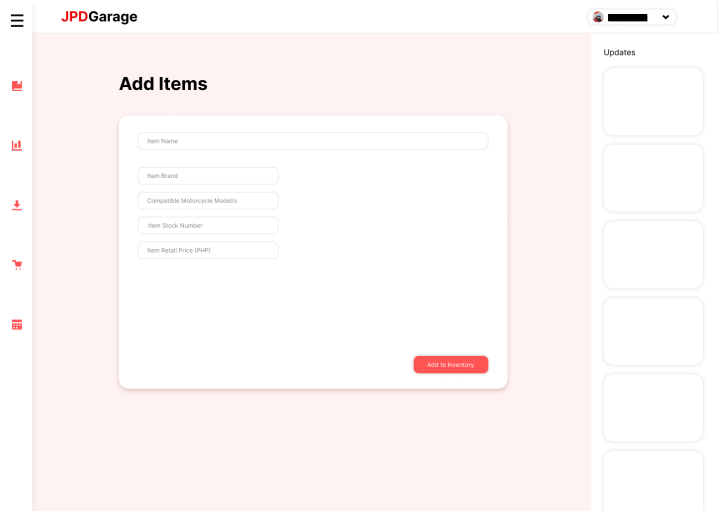


Figure 13. Add Items

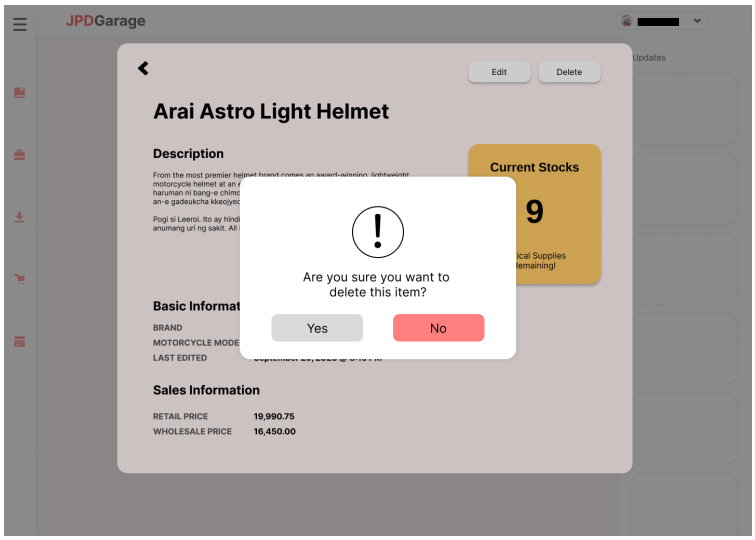


Figure 14. Delete Item

2.4.5. Sales and Payments

The layouts to see the status of the sales and payments. The information for sales is placed in an accordion with three groups: daily, monthly, and weekly. The groups can be seen at once or at the same time depending if the dropdown button of each group has been clicked or not. For the payments, there are two tabs: outgoing and incoming. This is to differentiate between the two categories of payments.

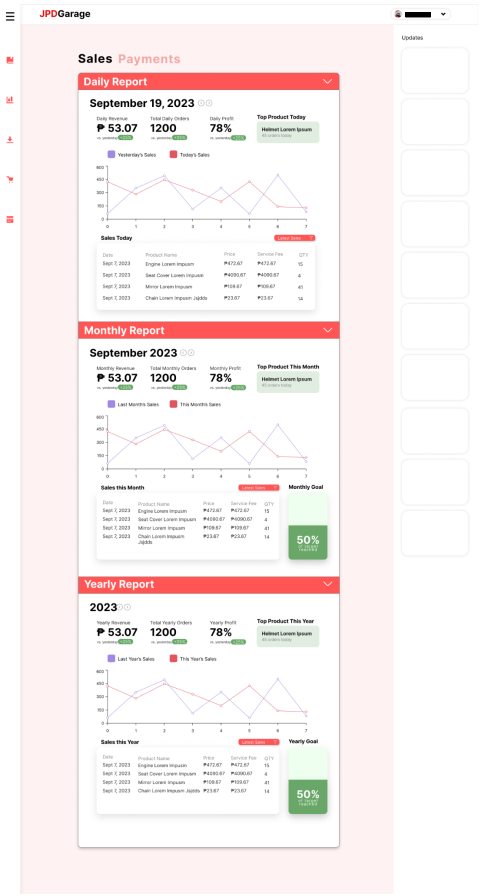


Figure 15. Sales Page for Daily, Monthly and Yearly Sales

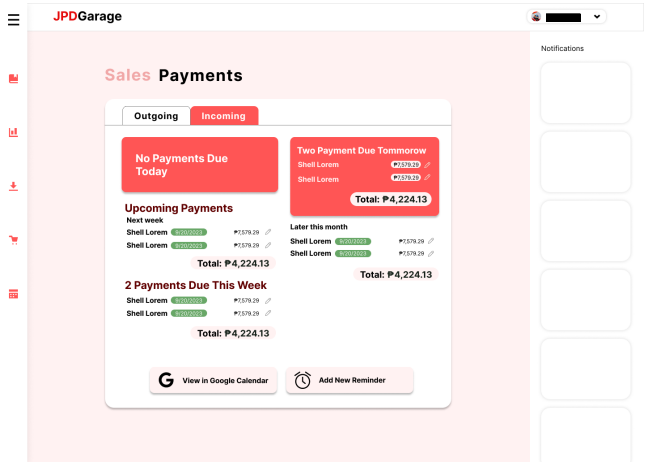


Figure 16. Incoming Payments

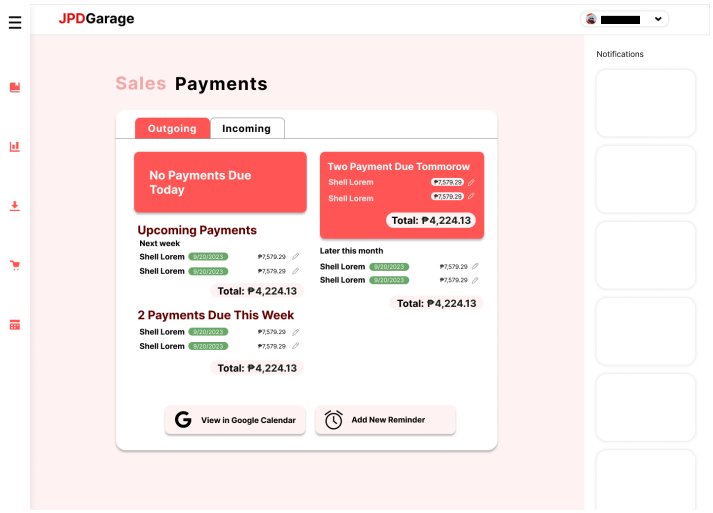


Figure 17. Outgoing Payments

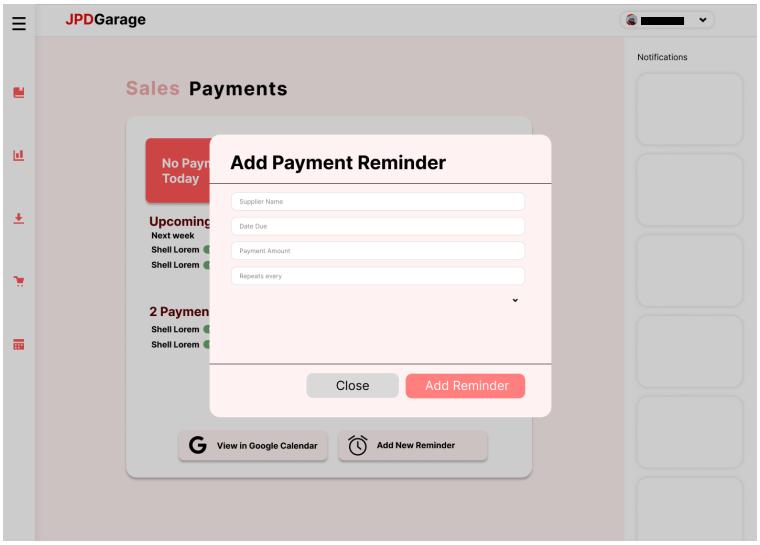


Figure 18. Add Payments

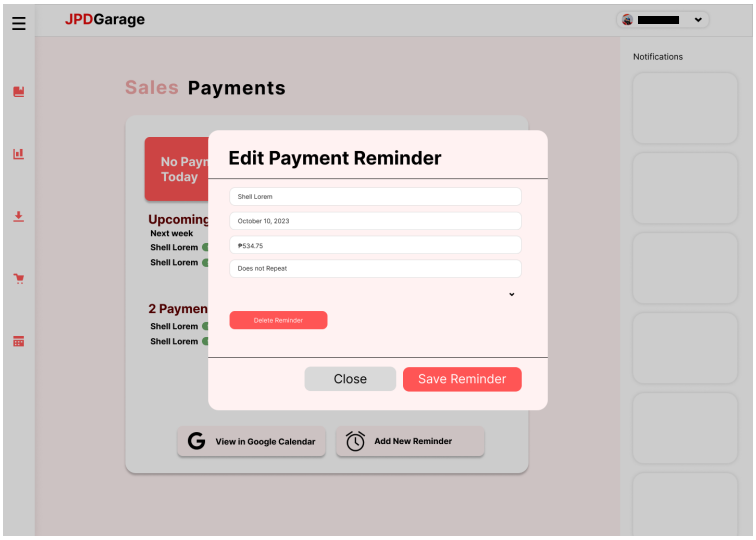


Figure 19. Edit Payments

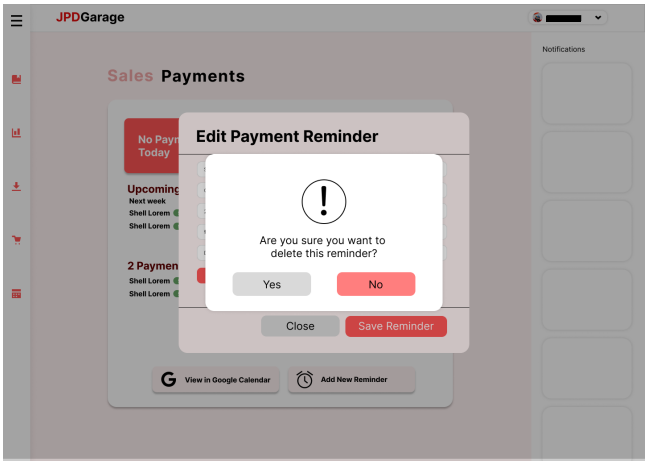


Figure 20. Delete Payments

2.4.6. Check Out

The layouts for the check out page or the shopping cart. The left side is to search for the item in the inventory and add it to the cart. The right side is the current items in the cart. There are checkboxes in order to delete multiple items. In which once a checkbox has been selected, the delete button would appear.

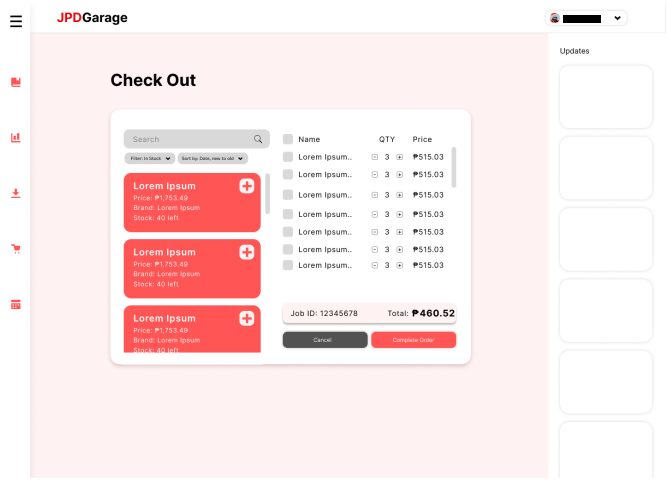


Figure 21. Check Out

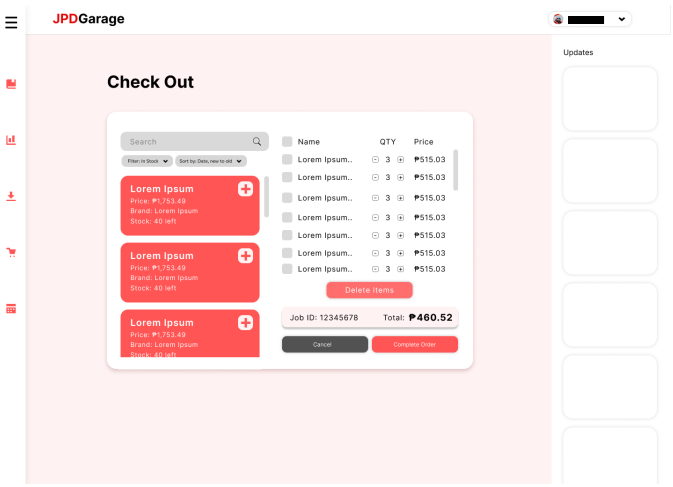


Figure 22. Select Items to Delete

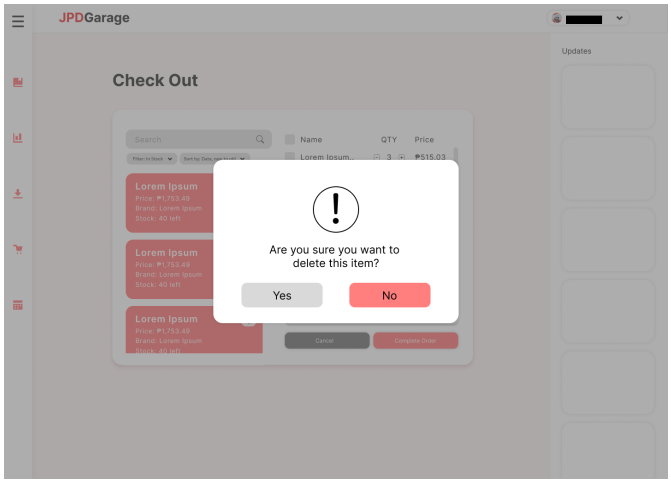


Figure 23. Delete Item Confirmation

2.4.7. Audit Log

The layout to see what kind of actions the users have done in the website.

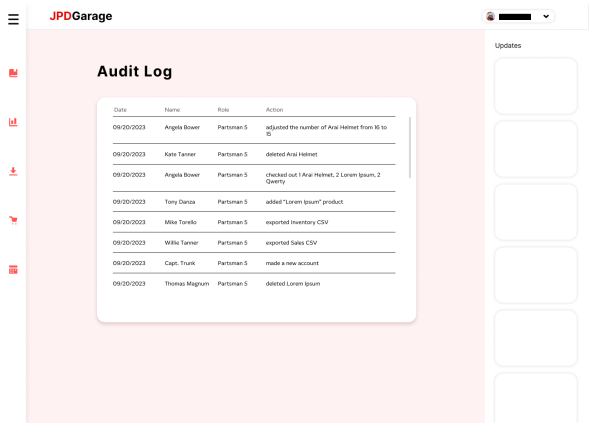


Figure 24. Audit Log

3. Feature Development

This section provides an in-depth exploration of our feature development process, highlighting the selected technology stack and the rationale behind choosing MERN Stack. It delves into each sprint iteration, outlining the specific features implemented during each iteration. The process followed in each sprint, including methodologies and tools used to provide a comprehensive understanding. Additionally, the challenges encountered and the strategies used to overcome them. This section serves as an overview of our feature development lifecycle, encapsulating the technological decisions, implementation progress, and the nuanced challenges faced throughout the project.

3.1. Tech Stack

Tech Stack: MERN (MongoDB, Express, React, Node)

The MERN stack is a popular and widely-used technology stack for building modern web applications. It is an acronym that represents four key technologies that seamlessly work together to enable the development of full-stack web applications.

MongoDB is a source-available, cross-platform NoSQL database program that utilizes JSON-like documents with optional schemas. Express.js, or Express, is a backend web application framework designed for building RESTful APIs with Node.js. React is a free and open-source frontend JavaScript library used for constructing user interfaces based on components. Node.js is a cross-platform, open-source JavaScript runtime environment that executes JavaScript code outside a web browser, running on the V8 JavaScript engine and supporting Windows, Linux, Unix, macOS, and more. Together, these technologies form the MERN stack, a powerful and versatile combination for developing modern web applications with a unified JavaScript-based approach across the entire stack.

Tech Stack Considerations

The team chose the MERN (MongoDB, Express.js, React.js, Node.js) stack for several reasons. Firstly, the development team is already well-versed in using this stack, which significantly accelerates the development process and ensures a smoother collaboration among team members. Secondly, the MERN stack aligns perfectly with the requirements of the client's inventory management system. Its flexibility, particularly the use of MongoDB for handling dynamic and evolving data structures, makes it highly applicable for efficiently managing inventory-related information.

Furthermore, the MERN stack remains a highly viable choice today due to its numerous advantages. The uniformity of JavaScript across the entire stack simplifies development, making it easier to maintain and scale applications. React.js provides a responsive and interactive user interface, crucial for an inventory management system's usability. Node.js ensures efficient backend operations, handling multiple concurrent requests effectively. MongoDB's scalability and adaptability to changing data structures make it suitable for evolving inventory needs. The stack's active community support and open-source nature contribute to its continued relevance, ensuring access to the latest enhancements and best practices in web development. Overall, the MERN stack not only leverages existing developer expertise but also offers a robust and modern foundation for building scalable and dynamic web applications.

Other Technologies Used

Server-Side Dependencies:

- bcrypt: ^5.1.1
- cors: ^2.8.5
- dotenv: ^16.3.1
- express: ^4.18.2
- jsonwebtoken: ^9.0.2
- moment: ^2.29.4

- mongoose: ^7.6.3
- nodemon: ^3.0.1
- validator: ^13.11.0

Client-Side Dependencies:

- **bootstrap:** ^5.3.2
- **react:** ^18.2.0
- **react-bootstrap:** ^2.9.1
- **chart.js:** ^4.4.0
- **cors:** ^2.8.5
- **lodash:** ^4.17.21
- **moment:** ^2.29.4
- **react-chartjs-2:** ^5.2.0
- **react-csv:** ^2.2.2
- **react-dom:** ^18.2.0
- **react-router-dom:** ^6.17.0
- **react-scripts:** 5.0.1
- **validator:** ^13.11.0

This project utilizes a range of server-side and client-side dependencies, including security-focused libraries (bcrypt), middleware for enabling cross-origin resource sharing (cors), environment variable management (dotenv), web application framework (express), token-based authentication (jsonwebtoken), date/time manipulation (moment), MongoDB object modeling (mongoose), server-side development tool (nodemon), and input validation (validator) on the server side, while the client side incorporates technologies for styling (bootstrap and react-bootstrap), building dynamic interfaces (React), data visualization (chart.js and react-chartjs-2), data handling (react-csv), routing (react-router-dom), and general project management (react-scripts).

3.2. Sprint Iteration 1

Chosen Features:

1. Skeletal Code
2. Edit - Inventory
3. Add - Inventory
4. Delete - Inventory
5. Search - Inventory

Process:

Essentially, a recurring process was followed for the development of features in this iteration and succeeding iterations. Front end developers would start by finishing some implementations of designs for the pages. Following this, the backend devs implement the backend features of finished designs while waiting for the other components and pages. Finally, feedback and insights would be received from the Scrum Master, Business Analyst, and Quality Assurance team. Naturally, bugs would often be identified by the QA team. This is why some time would be spent debugging and finalizing tasks right before presenting the results of the iteration to the product owner.

To start the project, a skeletal code was created for both the front and backend side. This includes the inventory schema and controller, as well as formats for basic react components and pages. For the inventory controller, functions were created for edit, add, and delete which were later made to work in the front end side of the program. Additionally, other useful functions were added like getInventoryItem for single items through their names and getInventoryItems for all items in the inventory. Finally, regex was used for the implementation of search.

Challenges:

For the backend developers, Postman was the API platform that was initially used to test and debug the backend features of the application.

For the actual features of the application, a common bug encountered by members of the quality assurance team was how information displayed in pages would not immediately update without actually refreshing the page. This affected the edit, delete, and add feature for items in the inventory. This issue was eventually addressed and resolved by the front end developers.

3.3. Sprint Iteration 2

Chosen Features:

1. Filter - Inventory
2. Login Page - Logging
3. Add to Cart - Sales
4. Remove from Cart - Sales
5. Shopping Navigation Tools - Sales
6. Inventory View - Sales
7. Purchase - Sales

Process:

For the filtering feature, this is very similar to search in the sense that it utilizes regex as well and it's actually merged with the search feature as well. However, instead we just added new request queries specifically for the filter input fields, motorModel and brand. For the login, we just made use of JWT Web tokens for our Authentication API, and implemented the necessary checks and hashing for the password and made sure that the token is stored in the local storage object. For the shopping cart feature, we split it into specific features and categorized them as sales since we deemed it necessary to split them to smaller, more manageable tasks so that it's easier to see the progression of the accomplished features.

Most shopping cart features are quite similar to that of the features of the main Inventory of the web application. A few miniscule logic adjustments were made however, for the Add to Cart, instead of just adding inventory items to the cart, it's finding the user by ID, and matching the carts based on the user. Moreover, since there are two types of transactions, retail and wholesale, the pricing shown would be based on whatever the user has chosen and would adjust accordingly when changed. The remove cart feature was fairly straight forward. We just store the IDs of selected items in an array, and then delete those from the cart with the filter() function. The Shopping Navigation Tools is basically exactly the same as the search/filter from the Inventory so we just reused the controller functions for that. Inventory View is also exactly the same as the inventory, except some visual changes such as no longer showing items with no stocks left, and a button that has been added to call for the Add to Cart controller function whenever it's clicked. Lastly, for the purchase feature, we just had a function in the schema that would deduct the corresponding inventory item's quantity based on its ID.

Challenges:

Some bugs were encountered and were also missing a few features. Specifically for the bugs, at the time of development, the total price of the cart was being computed but wasn't updating the price in the frontend side yet because the value wasn't being called there yet. Another bug was that it was only adding retail prices, not wholesale, regardless of whether we chose wholesale; we later realized this was because we realized that the status was not being saved to the database, only locally in a variable. These aforementioned bugs were fixed during the transition between Sprint 2 to Sprint 3. A challenge we also encountered was relating to the sessions for the login, when we figured out how the login worked, we encountered some issues when we were navigating through the pages. We later realized that there was a delay when the program is

verifying if the user is authenticated and logged in, thus we added a loading page that would conditionalize the pages so that it only enters the pages after verifying the user's status. Lastly, a challenge we encountered was despite most features working properly and even after fixing the bugs, we weren't able to implement the react context for the shopping cart thus making the web application not reactive, and would need the user to refresh the page to see the updated changes.

3.4. Sprint Iteration 3

Chosen Features:

1. Partsman & Admin Visibility - Logging
2. Summary - Sales

Process:

For the visibility features, we just added conditions to the App.js file checking if the user's role is "Admin" or "Partsman" and set the visibility accordingly. This was easy to do since the user's data was already being stored in the local storage. Moreover, for other features of the application, we also just added some conditions checking what the user's role is and set the appropriate visibility.

From what was implemented in Sprint 2, we further added a logic to the Confirm Order feature, where we were creating a new Order object whenever the user confirms their order. This new object would then be stored in the database through the save() function. Since we already have the orders, all we needed to do was display the items, and this is the Summary feature. For the Summary feature, we just fetched the orders based on the period, where the period (daily/monthly/yearly) is dependent on the component since the component (Daily/Monthly/YearlyReport.js) would be the one passing the period to the graph. Though, depending on the period, the data will first be aggregated so that for example, for the

monthly report, the orders per day in that month will be aggregated to produce the order points in the graph; this was also done for the yearly report. These all have corresponding functions and actually serve to also get the cumulative sales and total quantity count depending on the period of the sales.

Challenges:

There were no bugs from what the team knows, but that may not cover all edge cases. Moreover, a challenge we encountered from the devs team was that this was all unfamiliar to us, so it took quite some time figuring out the functionalities that can be done relating to react-chartjs. Lastly, given the time constraints we've had for this project, there was insufficient time to produce optimal code so it may be felt that the web application is a bit slow when loading/reflecting the data on the pages.

4. Quality Assurance and Testing

The team tested the web app in an agile way, following what the client wanted. To do this, the team used tools like Google Sheets, web browsers, and VSCode. The team divided the testing into three sprint iterations, each focusing on different features of the web app. For each feature, the team made and ran tests, and wrote down the results and problems. After each part, the team also made a working version of the web app and showed it to the client. The client gave the team their comments and suggestions, which the team used to fix the problems from the first part.

4.1. Test tools

Google Sheets

The team used Google Sheets to create and manage the test cases. Google Sheets is a web-based spreadsheet application that allows the team to easily organize, edit, and share data. The team chose Google Sheets because it is straightforward and easy to understand, and it supports features such as rows and columns for formatting, and team collaboration. Since tables are already easily understood, users could easily interpret the test cases. The team also used Google Sheets to track the status of our test cases, such as passed, failed, or pending.

Web Browsers

The team tested the web application using two different web browsers: Google Chrome and Microsoft Edge. These are two of the most popular and widely used browsers in the world, and they support the latest web standards and technologies. The team wanted to ensure that the web application works correctly and consistently across different browsers and platforms.

Visual Studio Code

The team used Visual Studio Code (VSCode) to launch the web application locally for faster testing without waiting for the main release. VSCode is a powerful and versatile code editor that supports various languages, extensions, and integrations. The team used VSCode to access the codebase of the web application from the developer branch in Git, which is a version control system that allows the team to collaborate and manage the code. The team also used VSCode to run and test the web application on local machines, and to make any necessary changes or fixes.

4.2. Test Approach

In this section, we explain the test approach that the team followed to ensure the quality and functionality of the web application. Our test approach was based on the acceptance criteria that we received from the client, which defined the expected behavior and outcomes of each feature.

Acceptance Criteria

The acceptance criteria were the main source of information and guidance for the testing process. It described the user stories, scenarios, and requirements that the web application should fulfill. The acceptance criteria were written in a clear and concise language, and it covered the following aspects:

- The purpose and goal of each feature
- The usability expectations of each feature

Test Cases

For each feature, the team dissected the acceptance criteria and converted them into specific technical test cases. The test cases were designed to verify and validate the web application against the acceptance criteria. The test cases included the following elements:

- The test case ID and title
- The test case description and objective
- The preconditions and postconditions of each feature
- The inputs, outputs, and environment
- The test case steps and expected results
- The test case status
- Comments from the QA

The test cases were created and managed using Google Sheets, as described in the previous section.

Once all test cases have passed in a feature, the QA will deem the feature as done.

4.3. Sprint Iteration 1

The first SPRINT iteration focused on testing the core features of the inventory system, as well as the account creation feature.

The features that the team tested in the first SPRINT iteration were:

- **Adding**: This feature allows the user to add new items to the inventory system, by entering the item name, brand, quantity, motorcycle model, and price.
- **Edit**: This feature allows the user to edit the details of an existing item in the inventory system, by selecting the item and modifying the fields.
- **Deleting**: This feature allows the user to delete an existing item from the inventory system, by selecting the item and confirming the action.
- **Searching**: This feature allows the user to search for an item in the inventory system, by entering a keyword of the item name and motorcycle model.

- **Filter**: This feature allows the user to filter the items in the inventory system, by choosing a category from the drop-down menu. This includes the motorcycle model, brand, and stock status.
- **Account Creation**: This feature allows the user to create an account for the inventory system, by entering an email, password, and role.

The team executed the test cases for each feature using the web browsers and VSCode, as described in the previous section. The team reported the results using Google Sheets, as described in the previous section. The team also used the developer tools and console features of the web browsers to inspect and debug the web application.

The testing results of the first SPRINT iteration were as follows:

- **Total test cases**: 114
- **Passed test cases**: 100
- **Failed test cases**: 4
- **Pending test cases**: 10
- **Pass rate**: 88%

After the initial testing, the team produced a working software that met the essential requirements of the client. The working software was deployed to the staging environment, where the client could review and provide feedback. The working software had the following features:

- The user can add, edit, delete, search, and filter items in the inventory system.

The working software had the following limitations:

- The adding feature does not validate the item name, allowing the user to enter blank or duplicate names.

- The deleting feature does not prompt the user for confirmation, leading to accidental deletions.
- The creation of an account for the inventory system, with a username, password, email, and role was not fully implemented.

The team planned to fix these bugs in the next iterations, as well as to add more features and enhancements to the web application.

As of finishing the three Sprint iterations, all the bugs are fixed and all test cases have passed.

4.4 Sprint Iteration 2

The second SPRINT iteration focused on testing the features related to the Point of Sale (POS) system, as well as fixing the bugs from the previous iteration.

The features that the team tested in the second SPRINT iteration were:

- **Add to Cart**: This feature allows the user to add items to the cart.
- **Remove from Cart**: This feature allows the user to remove items from the cart, by selecting item(s) from the cart list.
- **Shopping Navigation Tools**: This feature allows the user to navigate the add to cart and remove from cart such as searching for an item to add
- **User Log-In**: This feature allows the user to log in to the POS system, by entering the username and password.
- **Final Confirmation for Purchasing**: This feature allows the user to confirm the purchase.
- **Sales Inventory View**: This feature allows the user to view the sales history.

The team executed the test cases for each feature using the web browsers and VSCode, as described in the previous section. The team

reported the results using Google Sheets, as described in the previous section. The team also used the developer tools and console features of the web browsers to inspect and debug the web application.

The testing results of the second SPRINT iteration were as follows:

- **Total test cases**: 86
- **Passed test cases**: 78
- **Failed test cases**: 8
- **Pending test cases**: 0
- **Pass rate**: 91%

After the initial testing, the team produced a working software that met the essential requirements of the client. The working software was deployed to the staging environment, where the client could review and provide feedback. The working software had the following features:

- The user can add, edit, delete, search, and filter items in the inventory.
- The user can create an account for the inventory system, with a username, password, email, and role.
- The user can add, remove, and check out items in the cart.
- The user can log in to the POS system, with an email and password.
- The user can view the sales history.
- The user can confirm the purchase, by entering the payment method and the amount.

The working software had the following limitations:

- The add-to-cart feature does not update the total cost.
- There are no wholesale and retail modes.
- The final confirmation for purchasing feature does not clear the cart, leaving the items in the cart.

The team planned to fix these bugs in the next iterations, as well as to add more features and enhancements to the web application.

As of finishing the three Sprint iterations, all the bugs are fixed and all test cases have passed.

4.5 Sprint Iteration 3

The third SPRINT iteration focused on testing the visibility features of the different users, the summary of the sales and the bugs from the previous iteration.

The features that the team tested in the third SPRINT iteration were:

- **Partsman Visibility**: This feature limits the view of the partsman users to only the pages and features that they are allowed to access.
- **Admin Visibility**: This feature provides the administrator with access to all the pages and features in the inventory system.
- **Summary**: This feature shows the daily, monthly, and yearly report of the sales of the inventory including the revenue, total orders, and profit percentage.

The team executed the test cases for each feature using the web browsers and VSCode, as described in the previous section. The team reported the results using Google Sheets, as described in the previous section. The team also used the developer tools and console features of the web browsers to inspect and debug the web application.

The testing results of the third SPRINT iteration were as follows:

- **Total test cases**: 12
- **Passed test cases**: 12
- **Failed test cases**: 0

- **Pending test cases**: 0
- **Pass rate**: 100%

After the initial testing, the team produced a working software that met the essential requirements of the client. The working software was deployed to the staging environment, where the client could review and provide feedback. The working software had the following features:

- The user can add, edit, delete, search, and filter items in the inventory.
- The user can create an account for the inventory system, with a username, password, email, and role.
- The user can add, remove, and check out items in the cart.
- The user can log in to the POS system, with an email and password.
- The user can view the sales history.
- The user can confirm the purchase, by entering the payment method and the amount.
- The user's visibility is limited based on their credentials
- The user can see the summary of the sales in real time

The working software did not have any limitations.

5. Team Collaboration and Communication

This section discusses the processes and tools used in conducting the project. The project was conducted in less than 3 months, and every iteration, excluding Sprint 0, was done in less than 2 weeks.

5.1. Collaborative Tools

The team used a wide variety of tools including Notion, Google Drive, and Zoom. Notion was primarily used as the main project management tool. This contained the Team announcements, Scrum board, Meeting details, and links to other important tools. Notion was integral in conducting scrum activities as it became the all-in-one place for all the information regarding the project. Google Drive was then used to archive documents and other deliverables needed not only for the university's tracking but also for the product owner. Zoom was used to hold meetings for the Product Owner and team members as it allowed the ease of recording meetings.

The team also used other technical tools to help in creating the project. Figma is an online tool primarily used by the designers. It allowed the designers to collaboratively prototype and visualize the vision of the owner. Github was used as version control so that we ensure that changes in the code are tracked and saved. The IDE VScode was utilized for its Liveshare extension as it allowed members to collaboratively code.

5.2. Practices

The team conducted multiple scrum activities. During Sprint 0, Release Planning and Product Backlog refinement meetings were conducted to gain a better understanding of the project. On the subsequent sprints that followed, Sprint Plannings and Sprint Reviews were conducted with the Product Owner as both the beginning and end of each iteration. Within each iteration, team meetings were conducted via checkpoint meetings as opposed to daily standup meetings due to schedule conflicts.

For each iteration, it begins with the Product Owner and Developers to bargain which items to accomplish. The items are then worked on by both the developers and the designers. The progress on the items are tracked with the checkpoint meetings conducted. When issues arise, the Product Owner is contacted by the business analyst. After the items have been accomplished, the Quality Assurance team proceeds to check if the criterias formulated were passed. The end of the sprint is then decided at the Sprint Review where the Product Owner deems whether a feature is complete or not.

5.3. Product Owner and Client Communication

The team consistently plans meetings in advance, ensuring focused and productive discussions. Prior to concluding each session, the team proactively coordinates the next meeting with the product owner, specifying the location and date. This scheduling approach reflects a commitment to regular and structured communication, fostering a collaborative environment. Emphasizing the optimization of meeting duration, the team recognizes that a concise timeframe of at least 90 minutes strikes a balance between comprehensive discussions and respecting everyone's time. This systematic approach ensures that key project updates, requirements, and feedback are consistently addressed, contributing to a well-coordinated and effective development process. In instances where a predefined time or date for a meeting with the product owner proves challenging to establish, the team employs alternative communication methods to ensure continued progress. Specifically, the team utilizes messaging platforms to maintain regular updates and address queries promptly. Additionally, to provide a comprehensive overview of the testing phase, the team records video demonstrations of the discussed features. This recorded documentation serves as a valuable reference, allowing the product owner to visually assess the functionality of the implemented features and provide feedback.

6. Future Roadmap and Recommendations

This section includes what future software development teams may do for the project.

6.1 Unimplemented Features

Although the project is nearly finished, it was not completed due to the limited amount of sprints given to the team by the university. A major feature that was not implemented was an alert system. This alert system is divided into two parts: “Payment - Alert” and “Notification - Alert”. “Payment - Alert” would alert the admins that a payment is overdue, and it would continue to do so until it has been marked as resolved. This would also include payment alerts for those that the company needs to collect payments on. On the other hand, “Notification - Alert” alerts any user that a certain item is either low stock or out of stock. Lastly, an audit logging feature was also not implemented. This would track the employees and their actions on the website, and these actions that are logged includes: everything that was checked out, updating products, adding new products, exporting to CSV, deleting a product, making an account.

6.2 Recommendations

Due to the team’s limited knowledge and experience in software engineering, there are areas that allow for improvement on the project. Firstly, there was not a strict coding documentation in-place, and this may cause future developers to struggle in understanding the code. Next, a feature that may greatly enhance the business of JPD Garage, as suggested by the Product Owner himself, is a real-time barcode scanning system. Rather than manually adding items to the cart for check-in, orders from the shop that gets scanned are immediately reflected in the inventory and real-time sales page. Lastly, future developers may look into linking notifications received in the web application to the work emails of the employees. This may allow notifications to be received outside of the webpage, which greatly enhances its cause.